Compatibility, status, and extension

ar090 · 14 August 2026 · Draft

Contents

Developer guide

Compatibility

snnlang is additive. Historical commands and experiments use the legacy executor unless they explicitly select the graph executor. Bundles remain data-only, and *tools/snn* does not import the authoring package when replaying them.

Migration requires layered comparisons of structure, parameters, initialization, forward state, gradients, optimizer updates, checkpoints, resumed trajectories, learning curves, downstream measurements, and publication outputs. A successful smoke test does not establish equivalence.

Issue 73 is the active implementation and migration checklist. The independent exp022 gold-star campaign remains outside this documentation work.

Current support

Area	Status	Current boundary
Graph authoring	Implemented	Typed populations, inputs, parameters, projections, groups, outputs, and observables.
Bundles	Implemented	Canonical JSON, hashes, manifests, assets, reports, and diagrams.
Graph simulation	Implemented	Dense COBA-LIF and leaky-integrator graphs with AMPA/GABA projections.
Arbitrary topology	Implemented	Named feedforward, recurrent, and feedback projections with integral delays.
Runtime continuation	Implemented	Validated save, load, and continuation of dynamic graph state.
Readouts	Implemented	Mean voltage, final voltage, spike count, spike rate, cumulative potential, and valid-

		time masks execute through graph operations.
Input bindings	Partial	Resolved dense-array, valid-time-mask, and sparse event-stream bindings emit a versioned execution protocol; portable dataset loaders and encoders are pending.
Training recipes	Implemented	Validated objectives, regularizers, groups, gradient choices, duration, and reachability compile to canonical data.
Native training	Core implemented	Deterministic epoch/minibatch AdamW training and versioned named checkpoints support exact mid-epoch CPU resume; accelerator stochastic state and parity gates remain.
Collection migration	Not implemented	Requires the complete issue 73 equivalence process.

What remains

The following work is **not implemented**. Issue 73 tracks it as an active checklist.

Complete graph and protocol vocabulary

- Add portable dataset-loader and encoder bindings. Fixed and categorical variable-rate Poisson protocols are implemented.

The complete collection training vocabulary is implemented: disabled and trainable recurrent variants, initializer metadata, constraints and units, exhaustive parameter groups, gradient choices, spike-budget regularization, variable graph timesteps and presentation durations, differentiable-route validation, and element-level rejection are explicit.

- Record dataset identity, split, sample cap, batch size, shuffle behavior, and all stochastic seeds as execution protocol.

Graph-native training and checkpoints

Deterministic dataset iteration, named digest-bearing target bindings, versioned graph-training checkpoints, selected and final persistence, strict named loading, the one-layer legacy parameter map, and exact mid-epoch CPU optimizer/random-stream/data-order resume are implemented.

- Extend the checkpoint contract to accelerator stochastic state when accelerator training is validated.
- Validate COBA, PING, trainable recurrence, fine timesteps, variable rates, MNIST, SHD, and deeper trained graphs.

Inference and interventions

- Override inference duration, input rate, timestep, and supported projection strengths through an explicit execution contract.
- Provide hidden-spike deletion and Poisson-addition interventions without reaching into backend internals.

- Stabilize names and shapes for population spikes, membrane traces, rates, logits, accuracy, and rasters.
- Preserve seed-labelled caches and fail closed when a compiled graph cannot support an intervention.

Artifacts and campaigns

- Version the metric, checkpoint, and provenance schemas needed by collection campaigns.
- Integrate graph and training digests with runstore validation, resumption, promotion, archive, and restore.
- Select `legacy` or `graph` explicitly while keeping campaign ownership and publication behavior unchanged.

Conformance and migration

The first hand-checkable CPU cases prove exact named parameter tensors and complete E/I spike, membrane, AMPA, and GABA traces for both feedforward-isolated and actively recurrent one-layer PING networks built from one parameter set. Mean-voltage logits agree under the predeclared 10^{-6} absolute and relative CPU tolerance. The first case also fixed the legacy recurrent layer map to its actual one-based names and aligned the `MeanVoltage` default with the legacy 2 ms output membrane.

A four-update backward case makes all six feedforward and recurrent tensors trainable and compares the complete loss trajectory, every final named gradient, the constrained final parameters, and all named AdamW tensors under the same predeclared tolerance. A two-update checkpoint followed by two resumed updates is bit-identical to the uninterrupted graph trajectory. The fixture applies the legacy trainer's non-negative projection after each optimizer step, separating optimizer state from the constrained stored parameter value.

Bidirectional parameter interchange now imports and exports the supported legacy state keys with exact coverage, shape, dtype, and mapping-version provenance. Forward conformance is rerun after a graph $\rightarrow$ legacy $\rightarrow$ graph round trip. Optimizer interchange remains deliberately excluded because a legacy optimizer object does not satisfy the portable graph checkpoint contract.

- Compare topology, parameter tensors, initialized state, forward traces, loss, gradients, optimizer updates, checkpoint interchange, exact resume, learning trajectories, interventions, and aggregations.
- Define exact fields and numerical tolerances before viewing the final comparison.
- Run CPU and publication-accelerator conformance cases and record any limits on numerical determinism.
- After the independent legacy gold-star campaign is complete, run the full snnlang campaign under the frozen scientific protocol and compare every result, uncertainty band, claim, and figure.
- Retain both immutable campaigns and require deliberate review before changing the default executor or migrating the publication collection.

Extending snnlang

Add a feature when it represents reusable computation or execution. Keep it in an experiment runner when its meaning depends on one hypothesis or figure.

A capability should include a clear authoring API, versioned serialization, compiler validation, precise diagnostics, a hand-checkable execution fixture, explicit state and provenance semantics, and compatibility tests. Add an executable example once the feature is mature enough to teach.

Avoid opaque callbacks, access to backend internals, positional checkpoint guesses, and silent fallbacks.

API reference

Validation and diagnostics

```
snn.validate_graph(graph) -> ValidationResult
result.raise_for_errors() -> None
```

`ValidationResult` exposes `.diagnostics`, `.errors`, and `.warnings`. Each `Diagnostic` contains `severity`, `code`, `message`, and optional `subject`; `.line()` renders a stable one-line form.

Compilation raises graph and training errors. Target capability gaps remain diagnostics so a bundle can still describe computation unsupported by the selected backend.

Executor capability inspection

```
graph_capability_issues(graph) -> list[CapabilityIssue]
plan_graph(graph) -> GraphPlan
```

`CapabilityIssue` identifies the graph element, missing capability, and message. `plan_graph` performs execution-specific topology, shape, polarity, scheduling, and integral-delay validation before constructing a `GraphPlan`.

`GRAPH_CAPABILITIES_V1` is the current machine-readable executor boundary. It lists supported neuron, synapse, operation, connection, recording, delay, and training capabilities.

Conformance reports

```
policy = ComparisonPolicy(mode="numeric", atol=1e-6, rtol=1e-5)
report = compare_conformance_layers(
    case_id,
    reference,
    candidate,
    policies={"forward": {"logits": policy}},
)
report.require_passed()
write_conformance_report(path, report)
```

Reports use schema `tools/snn.conformance-report/v1`. Each layer and field is named explicitly. Missing fields, extra fields, shapes, dtypes, values, maximum absolute and relative errors, and the applied policy are recorded. Exact comparison is the default; numerical tolerance must be declared for a specific field, and an unused rule is rejected. `canonical_json_tensor` encodes topology and provenance structures for exact comparison beside numerical tensors.

Code map

- `tools/snnlang/core.py`: authoring objects and primitive specifications.
- `components.py`: reusable graph-building components.
- `readouts.py` and `ops.py`: standard operations.
- `training.py`: training recipes.
- `compiler.py`: validation and bundle writing.
- `visualize.py`: bundle reports.
- `tools/snn/execution.py`: graph planning and execution.
- `tools/snn/conformance.py`: versioned layered comparison reports.
- `tools/snn/bundle.py`: the narrow legacy adapter.
- `tools/snnlang/tests` and `tools/snn/tests/test_execution.py`: focused conformance fixtures.